PCP: Proof-Contingent Payment

Julian Sugiarto
github.com/juliansugiarto/pcp
22 October 2026

Abstract. A payment that settles if and only if the seller has proven, before payment, that the work is correct would let strangers trade without an escrow agent, a platform or a court. Bitcoin made payments irreversible and so protected sellers; it left buyers to "routine escrow mechanisms" that were never specified. We specify one. The seller encrypts the result under a fresh key k, proves in zero knowledge that the ciphertext decrypts under k to a result that passes the buyer's predicate and that SHA256(k) equals a Lightning payment hash, and the buyer pays an invoice to that hash. Claiming the payment reveals k, which is the decryption key, so neither side can hold the other's part without giving up its own. Every message is bound by hash to an offer, a buyer nonce and a backend identifier, so a proof cannot be replayed or re-priced. Every agent acts under a human-signed mandate with a scope, a cap and an expiry, and is paid into its principal's node, so software can trade without being able to fund itself. Only proof systems with no trusted setup are accepted, which removes the known attack on the 2016 construction. A full sale with a real zkVM proof has settled between two real Lightning nodes on regtest in continuous integration: 151 s to prove a 9x9 sudoku on 4 vCPU, 0.018 s to verify, 224 KB receipt, key in the buyer's hands 0.53 s after paying. Nothing here is audited.

1. Introduction

Nakamoto's introduction contains two halves of a sentence: "Transactions that are computationally impractical to reverse would protect sellers from fraud, and routine escrow mechanisms could easily be implemented to protect buyers" [1]. Bitcoin delivered the first half. The second was never written down as a protocol. A buyer who pays a stranger in bitcoin for a piece of work is protected by nothing but the stranger's goodwill, and the usual repair is a platform in the middle that holds the money, judges the work, takes a fee and can be ordered to freeze either.

Exchange between strangers needs one of two things: simultaneity, so that neither side moves first, or an intermediary who moves for both. Intermediaries are what Bitcoin was built to remove. Simultaneity is what this paper is about.

Any trade between strangers carries three risks: (1) the counterparty does not deliver or does not pay; (2) the good is not what was promised; (3) the seller's right to the good is not clean, and a third party takes it back later. Proof-contingent payment (PCP) removes risk 1 entirely. It removes risk 2 when the promise can be written as a predicate that a program can check. It does nothing about risk 3, and neither can any proof system, because the facts that decide it live outside the trade.

The mechanism is not new. Maxwell described zero-knowledge contingent payment (ZKCP) in 2011 [2], and in February 2016 Maxwell bought a sudoku solution from Bowe for 0.10 BTC on the Bitcoin mainnet, live at Financial Cryptography [3, 4]. In 2017 Campanelli, Gennaro, Goldfeder and Nizzardo showed that the 2016 construction leaks part of the good to a buyer who generates the proof system's common reference string maliciously [5]. What this paper adds is a written protocol that binds proof, offer, buyer and invoice together; a mandate layer so that software agents can trade under a human's limits; a rule that admits only proof systems with nothing to poison; and measured numbers from a sale that settled over real Lightning nodes.

2. Proof-contingent payment

Fix a job class: a deterministic, total predicate check(input, R) with a canonical byte encoding for input and R. The buyer supplies input. The seller computes R, draws a fresh 32-byte key k from a CSPRNG, and computes

```
C = Enc_k(R)
H = SHA256(k)
```

The seller then produces a proof π of the statement

```
∃ R, k :  check(input, R) = true  ∧  C = Enc_k(R)  ∧  H = SHA256(k)        (1)
```

and sends (C, H, π) to the buyer. The buyer verifies π. If it verifies, the buyer pays a Lightning invoice whose payment hash is H. Lightning's hash time-locked contract (HTLC) enforces exactly one relation: the payee receives the funds only by revealing a preimage k' with SHA256(k') = H before the lock expires, and that preimage is returned to the payer as proof of payment [6, 7]. Since SHA-256 is second-preimage resistant, k' = k, and the buyer decrypts C.

The key k plays two roles at once: decryption key and payment preimage. That is the whole construction. Statement (1) says the key whose hash is on the invoice is the key that opens the ciphertext, and that what it opens is a correct result. The HTLC says the key leaves the seller only when the money arrives. No payment without k; no k, through the payment, without payment.

Atomicity follows from three properties. Soundness of π: the seller cannot prove a C that decrypts to a wrong result, or an H that is not the hash of the key that decrypts C. Zero knowledge of π: the buyer learns nothing about R before paying, so a buyer who walks away gains nothing but the seller's time. The HTLC: funds and preimage move in one step, or the funds return at the timeout.

Two things are deliberately absent. There is no third role: no escrow, arbiter, relay, coordinator or issuer; a Lightning routing node on the path is part of the rail, not of the protocol. And there is no on-chain verifier: the buyer verifies π on its own machine, so the

proof need not be cheap on a blockchain or wrapped in a pairing-based proof for that purpose.

3. Protocol

PCP/0 is the first version. All identifiers are tagged hashes over length-prefixed fields, never over JSON text:

```
lp(x)                = u32_be(len(x)) || x
tagged_hash(t, f1..)  = SHA256( lp("PCP/0/" || t) || lp(f1) || lp(f2) || ... )
```

The one exception is H = SHA256(k), kept plain so that any Lightning node can use it. The derived values that carry the argument are:

```
spec_hash        = tagged_hash("spec", [job_id, params])
input_commitment = tagged_hash("input", [input])
offer_id         = tagged_hash("offer", [version, job_id, spec_hash, input_commitment,
                                buyer_nonce, price_msat, offer_expires_at, seller_node,
                                backend_id, enc_scheme, rail])
ciphertext_hash  = tagged_hash("ciphertext", [C])
statement_hash   = tagged_hash("statement", [version, job_id, spec_hash, input_commitment,
                                offer_id, buyer_nonce, ciphertext_hash, payment_hash])
```

A sale is five messages.

Request (buyer to seller): job_id, spec_hash, input, input_commitment, a fresh 32-byte buyer_nonce, max_price_msat, the buyer's mandate (section 4) and an Ed25519 signature by the buyer's agent key over the request.

Offer (seller to buyer): the request's job fields echoed, price_msat, offer_expires_at, seller_node, backend_id, enc_scheme, rail, the seller's mandate and the agent's signature. offer_id commits to every term. Because backend_id is one of the terms, an offer names the exact proof system and, for a zkVM, the exact guest binary (section 5). An expired offer is void: the seller refuses to invoice on one and the buyer refuses to pay on one. The seller also fulfils only an offer it issued, unchanged: it checks its own agent signature over offer_id, so an offer handed back with a lower price or a later expiry is refused.

Proof bundle (seller to buyer): backend_id, the statement {version, job_id, spec_hash, input_commitment, offer_id, buyer_nonce, ciphertext_hash, payment_hash}, the ciphertext C and the proof π. The statement is the public input to the proof. It contains offer_id and buyer_nonce, so π is a proof about this offer to this buyer and nothing else.

Invoice binding (seller to buyer): offer_id, statement_hash, and a BOLT 11 invoice [7] whose p tag (payment hash) is H, whose h tag (description hash) is statement_hash, whose amount is price_msat and whose payee, recovered from the invoice signature, is seller_node. The description hash is the second binding: BOLT 11 reserves a 256-bit field for the hash of a long description, and PCP fills it with the hash of the statement that was proven, so the invoice commits to the proof it pays for.

Settlement. The buyer pays. The seller's node holds the HTLC until the seller settles it with k; LND's hold invoices provide this [8]. The buyer receives k as the payment preimage, checks SHA256(k) = H, decrypts C under offer_id, and re-runs check(input, R) locally if it is cheap. Both sides sign a receipt (offer_id, statement_hash, bundle_id, payment_hash, preimage, amount, time, both mandate ids) and append it, with the bundle and the input, to their principal's log, so the principal can re-verify the sale offline.

The buyer's verification order is fixed, and a failure at any step aborts the purchase: check the offer (job, commitment, nonce, price, expiry, backend, mandates); check the bundle's version, nonce, offer_id and ciphertext hash; verify π against a statement the buyer builds from its own state, taking only ciphertext_hash and payment_hash from the bundle; check that the invoice names this offer and statement, that its payment hash is the proven H, that amount, payee and description hash match, and that it leaves enough time to pay without too long a lock; only then pay. Everything before payment is offline.

Encryption in the reference backend is ChaCha20-Poly1305 [9] under a key derived from k by a tagged hash, with a zero nonce (safe because the key is used once) and offer_id as associated data, so a ciphertext made for one offer does not open under another. Encryption is deterministic given (k, R, offer_id), which is how the prover recomputes C inside the proof.

4. Mandates

An agent is a program, or a person at a client, that plays buyer or seller. Every agent acts for a principal: a human, or an organisation a human controls. The principal signs, offline, a **mandate** for the agent's key:

```
mandate = {version, principal_key, agent_key, role, scope, cap_msat,
           per_deal_max_msat, payout_node, expires_at, nonce}
mandate_id   = tagged_hash("mandate", [fields above])
principal_sig = Ed25519(principal_sk, mandate_id)
```

scope is the list of spec_hash values the agent may trade; cap_msat is a cumulative limit over the mandate's life; per_deal_max_msat the largest single deal; payout_node the principal's Lightning node; expires_at an absolute time. Every request and every offer carries the sender's mandate and a signature by agent_key over the message, so a copied mandate is useless without the agent key.

Each side checks the other's mandate and its own before trading: present, verifies, right role, unexpired, covers the job class, price

within the per-deal maximum, cap not exhausted. The seller's invoice must pay the principal's payout_node, and the buyer checks this twice: seller_node = payout_node when accepting the offer, and invoice payee = payout_node before paying. This is the most important line of the design. An agent paid into its own node funds itself, and a self-funding agent is no longer stopped by its principal's cap.

An agent cannot be its own principal: principal_key = agent_key is refused by the agent's own client and by every counterparty. The protocol can check only the inequality, not who holds which key; an agent given a second key and told to call it its principal is not detectable. A mandate proves that a key authorised a key within limits. It does not prove that a human holds the key, and it does not prove identity in law.

There is no issuer. Any Ed25519 key [10] is an acceptable principal key, a mandate is verifiable with no network access and no third party, and an implementation must not require any particular issuer's credential. The stop button is the principal's: let the mandate expire, lower the cap, or close the channel that funds the agent. A channel funded with N sat caps the agent at N whatever any client does.

5. Proof backends

A backend implements prove and verify for statement (1), is named by backend_id, and declares is_secure. The buyer refuses any backend it does not run, and refuses an insecure one unless explicitly configured for testing.

Transparent only. The 2017 attack [5] works because the buyer in pay-to-sudoku generated the zk-SNARK common reference string, and a malicious string breaks zero knowledge: the buyer learns a sudoku cell without paying. PCP/0 requires backends whose proofs have no trusted setup and no reference string that either party chooses. Hash-based STARKs and inner-product-argument systems over hash-to-curve parameters both qualify. The premise of the attack is then absent rather than mitigated. The Groth16 wrapper that zkVMs offer for on-chain verification re-introduces a setup; PCP/0 verifies off-chain and must not require it.

zkVM backend (RISC Zero). The relation (1) is one pure function, statement::evaluate, in a no_std crate whose only dependencies are SHA-256 and ChaCha20-Poly1305. The dev backend calls it in the clear; the RISC Zero guest [11] calls it inside the zkVM. The guest reads the private input (spec_hash, input, R, k, C, offer_id, buyer_nonce), checks the predicate and the encryption, and commits the canonical statement bytes as its journal, so SHA256(journal) = statement_hash. The receipt is a succinct STARK; the buyer rejects dev-mode receipts and verifies against a pinned image id. A binary cannot contain its own image id, so the id goes into backend_id as risc0-zkvm-3-succinct/, which offer_id commits to. The guest is built in Docker so that the id is reproducible: two independent CI builds produced abe2df2125017d6785cab6fe7103c0ecf3fc130e34d9f7efa0359c8ffe06d941, and a buyer verifies against the id it built itself. Proving is local by construction (no remote-proving feature is compiled in), since the guest input contains the goods.

halo2 backend. The same statement as a halo2 circuit [12] over the Pasta curves with IPA commitments: 9x9 sudoku only, a Poseidon [13] keystream cipher (zk-poseidon-ctr/0) because an AEAD is too expensive in-circuit, and one SHA-256 compression for H as a bit-level chip written in this project (section 6). Parameters are derived by hash-to-curve and the verifier computes the verifying key itself; it never accepts one from the seller. The circuit is labelled reviewed inside the project, not audited, and its is_secure is false: a buyer must opt in by policy, as with the dev backend. It is published as a target to break, not as the backend to pay through.

Backend agility. Since backend_id and enc_scheme are bound into offer_id, a buyer's policy can accept or refuse any backend by name, and a future backend is a new identifier, not a new protocol.

6. Security analysis

What the seller cannot do. Send a C of a wrong result, of something other than the proven result, or swapped after proving: the proof relation and the ciphertext hash catch all three. Invoice to a hash whose preimage does not open C: the buyer checks the invoice's hash against the proven H before paying. Alter amount, payee or description: checked. Reveal a wrong preimage: the HTLC rejects it and the funds return. Replay a bundle or invoice made for one buyer to another: the statement carries buyer_nonce and offer_id, C is bound to offer_id, the invoice to statement_hash, and k is fresh per sale. Settle so late that the refund races the settlement: the seller withholds k within a margin of the HTLC timeout, and cancels the HTLC so the buyer's funds return at once rather than at expiry.

What the buyer cannot do. Learn R from (C, H, π) before paying: zero knowledge. Pay and not receive k: the HTLC releases funds only against k. Poison the proof system's parameters: there are none. What the buyer can do is walk away after the seller has proven, costing the seller its work and proving time and nothing else. Mitigations are outside the protocol: price proving in, rate-limit unknown buyers, or the contingent-service variant of [5].

What each side can lose, by step. Before verification: the buyer nothing, the seller work and proving cost. Before payment: the buyer verification time. With the HTLC locked: the buyer's price, until settlement or timeout. On settlement: the buyer has paid and has R; the seller has the money. On timeout: the buyer is refunded, with liquidity tied up for the lock period. There is no row in which one side has the other's part without giving up its own.

Timelocks. The seller proves before it invoices, so proving time counts against the offer lifetime, not the invoice or the HTLC. The buyer refuses an invoice with too little time left to pay, or one that demands a lock longer than its maximum. On Lightning the

reference presets are in whole blocks: the seller asks for a final CLTV of 24 blocks, the minimum LND v0.21 accepts for a hold invoice; the buyer accepts a lock of up to 24 + 2 x 80 + 20 = 204 blocks, room for two routing hops at LND's default per-hop delta of 80; and the seller settles only with more than 14 blocks left, above LND's own hold-expiry cancel of 12 blocks. In the regtest sale the invoice expiry was 600 s and the HTLC was accepted at block 113 with expiry at block 140. Routing through intermediate nodes has not yet been tested.

Replay and the log. Buyers should refuse any H they have seen before; the reference client does not yet do this. The principal's log holds signed receipts and signed refusals with reason codes; altered or forged entries are detectable, omission by a modified client is not. The principal's node history and the counterparty's receipt, which names this agent's mandate, are the evidence that remains.

Assumptions. SHA-256 collision and preimage resistance; the AEAD and key derivation; the proof system's soundness and zero knowledge under its own assumptions (hash-based for STARKs, discrete log on Pasta at about 126 bits for halo2); Ed25519 for mandates; and the rail: routing nodes can delay or fail a payment but cannot settle without k. Every hop on the route learns k when the payment settles, so confidentiality of R after the sale rests on C not being public: the bundle goes to the buyer only. Hops also see H and can correlate; PTLCs would remove that.

A failure, caught before anyone paid. The halo2 backend's first SHA-256 was the Table16 gadget from halo2_gadgets 0.5.0. An internal read-only review on 21 October 2026 found four independent ways for a seller to forge H: free round constants, unbounded carries, message-schedule words W_16 to W_63 not linked to their computation, and free final-round state. Two were reproduced with MockProver: a seller could invoice to SHA256(p), encrypt under k ≠ p, prove, be paid, and leave the buyer with a preimage that opens nothing. Upstream had removed the gadget three weeks earlier "due to multiple soundness vulnerabilities", the carry issue "confirmed exploitable" [14]; the project knew the module was gone and had not read why. It was replaced by a bit-level chip in which every advice cell has exactly one owner, constants and IV are in fixed columns, every carry is range-constrained, and schedule words, round inputs and the digest are bound by copy constraints. It is tested with 19 malicious witnesses and 14 compensated forgeries, in which the trace downstream of a tampered cell is recomputed and H is set to the forged digest so that only one constraint can reject it. Mutation testing showed the 19 original tests could not detect two Table16 flaw classes being reinstalled; the compensated tests could. A second internal review found no soundness issue in the chip. The same bug class sat in audited production code elsewhere for four years (CVE-2026-54496 [15]). The chip remains unaudited, is published as a target with a soundness bounty and a privacy bounty, and the zkVM backend is the one meant for payments.

7. Measurements

All numbers are from 21 October 2026, on regtest or a simulated rail, in continuous integration. Nothing has run on signet or mainnet.

A real sale: real proof, real Lightning nodes. GitHub-hosted ubuntu-latest runner, 4 vCPU x86_64, 16 GB RAM, no GPU; risc0 3.0.6; bitcoind 28.0 regtest; two LND v0.21.4 nodes; Docker-built guest with image id abe2df21…d941; backend risc0-zkvm-3-succinct, rail lightning-bolt11/0; price 1,000,000 msat. Both sales ended SETTLED, the seller's channel balance rose by exactly 1,000 sat per sale, and SHA-256 of the revealed preimage equalled the invoice's payment hash.

Job	User cycles	Total cycles	Prove	Verify	Receipt	Peak memory	Buyer has k after paying
pcp/sudoku/v0, n = 9	98,941	262,144	150.6 s	0.018 s	223,748 B	2.36 GB	0.53 s
pcp/testsuite-toyexpr/v0	114,019	262,144	149.7 s	0.018 s	223,770 B	2.36 GB	1.03 s

Proving time follows the padded cycle count, not the job: both jobs pad to 2^18 cycles and cost the same. Of the 600 s offer lifetime, 151 s was used. The earlier native (non-reproducible) guest padded the sudoku to 2^17 cycles and proved in 64.6 s with 1.41 GB on the same class of machine; the reproducible build is the one a stranger can check, so the larger numbers are the ones that count. Bringing the guest back under 2^17 cycles would roughly halve them.

halo2 backend. Different machine: 2 CPUs, 8 GB RAM (about 6.3 GB usable), rustc 1.97.0, release build, one prover at a time.

Step	Time	Peak memory
Transparent parameters, k = 13, from scratch (once per machine)	4.6 s	5 MB
Verifying key computed by the buyer (once per circuit version)	0.81 s	21 MB
Proving, per sale	7.8 s	572 MB
Verification, per sale	0.057 s	22 MB

The proof is 15,936 bytes; the public inputs 6,217 bytes. The replacement SHA-256 chip removed a 2^16-row lookup table, so k fell from 17 to 13 and proving fell from 143.6 s and 4.29 GB to the figures above; the proof grew from 5,088 bytes because IPA proof size scales with the 80 advice columns the bit-level design uses.

The two backends are not measured on the same machine and the comparison is rough: the STARK receipt is about 14 times larger

than the halo2 proof and verifies faster; proving is slower; in exchange the SHA-256 is RISC Zero's circuit rather than one this project wrote, and the guest is plain Rust.

8. Limits

Checkable goods only. The predicate is the contract. A result that passes check is a correct delivery by definition, even if it is not what the buyer had in mind. In scope: pure computation with a checkable output, schema-valid data, code that passes a hermetic test suite. Out of scope: judgement work (any judge is a trusted third party), physical delivery (needs an oracle), large-model inference (proving is not viable; attestation re-introduces a vendor), and services with no good to reveal, which need the contingent-service variant. A local registry of reviewed job classes is enforced by default; registered means reviewed and tested, not safe. Some checkable classes are excluded by policy: third parties' secrets or credentials, exploits, personal data, physical actions. A fork can define any class; that is inherent.

Third-party and title risk. PCP removes risk 1 and, for checkable promises, risk 2 of section 1. It does nothing about risk 3.

Regtest only. The real-node results are from two LND nodes on regtest in CI. No signet, no mainnet; no stranger has yet run the verifier. Whether LND macaroon constraints suffice as an agent's credential on the real rail is not yet checked. The reference client keeps its state (open sales and their keys, the cap book, receipts) in process memory; a restart in mid-sale loses it, and persistence is the next step. The default buyer policy caps a deal at 10,000 sat: PCP/0 is for small, repeatable jobs, and a settled payment is final.

Not quantum-resistant as a whole. A large quantum computer breaks elliptic-curve discrete logarithms: Lightning and Bitcoin signatures (a bigger and older target), Ed25519 mandates, and the soundness of discrete-log proof systems including the halo2 backend, so such a seller could be paid for nothing. What holds, as far as is known: SHA-256 for H and the hash-based soundness of STARKs without a Groth16 wrapper, at about 128 bits for a 256-bit key under Grover. Mitigation is agility: a buyer policy can refuse discrete-log backends by backend_id, and mandates can move to a post-quantum signature in a later version. Settlement cannot be more quantum-resistant than the rail it settles on.

Not audited. The specification, the Rust implementation (66 tests, 73 with the LND rail, plus 4 run against real LND nodes), the zkVM guest and the halo2 circuit have been reviewed and tested inside the project. Nobody outside it has audited any of them. None of the rules in the specification is enforceable against a modified client; they are defaults, enforced by counterparties' refusal.

9. Conclusion

We have specified the half of Nakamoto's sentence that was left as an exercise. The construction is ZKCP with three additions: a binding discipline that ties proof, offer, buyer and invoice by hash so that nothing can be replayed or re-priced; a mandate layer that lets a program trade under a human's signed limits and be paid only into that human's node; and a rule that admits only transparent proof systems, so that the known attack has nothing to poison. The result has settled a sale with a real STARK proof between two real Lightning nodes, on regtest, with the buyer verifying before paying and holding the key half a second after. The costs are a few minutes of CPU for the seller and milliseconds for the buyer. What remains is what no author can supply alone: an audit, a stranger who runs the verifier, and a sale on a network where the money is real.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. https://bitcoin.org/bitcoin.pdf

[2] G. Maxwell, "Zero Knowledge Contingent Payment," Bitcoin Wiki, 2011. https://en.bitcoin.it/wiki/Zero_Knowledge_Contingent_Payment

[3] G. Maxwell, "The first successful Zero-Knowledge Contingent Payment," Bitcoin Core blog, 26 February 2016. https://bitcoincore.org/en/2016/02/26/zero-knowledge-contingent-payments-announcement/

[4] S. Bowe, "pay-to-sudoku," presented at Financial Cryptography 2016. https://github.com/zcash-hackworks/pay-to-sudoku

[5] M. Campanelli, R. Gennaro, S. Goldfeder, L. Nizzardo, "Zero-Knowledge Contingent Payments Revisited: Attacks and Payments for Services," ACM CCS 2017. IACR ePrint 2017/566. https://eprint.iacr.org/2017/566

[6] Lightning Network specifications, "BOLT #3: Bitcoin Transaction and Script Formats" (HTLC outputs). https://github.com/lightning/bolts/blob/master/03-transactions.md

[7] Lightning Network specifications, "BOLT #11: Invoice Protocol for Lightning Payments." https://github.com/lightning/bolts/blob/master/11-payment-encoding.md

[8] Lightning Labs, LND API reference, "AddHoldInvoice." https://lightning.engineering/api-docs/api/lnd/invoices/add-hold-invoice/

[9] Y. Nir, A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, 2018. https://www.rfc-editor.org/rfc/rfc8439

[10] S. Josefsson, I. Liusvaara, "Edwards-Curve Digital Signature Algorithm (EdDSA)," RFC 8032, 2017. https://www.rfc-editor.org/rfc/rfc8032

[11] RISC Zero, zkVM developer documentation, "Receipts 101." https://dev.risczero.com/api/zkvm/receipts . Toolchain used: risc0 3.0.6.

[12] Zcash, "The halo2 Book." https://zcash.github.io/halo2/ . Underlying construction: S. Bowe, J. Grigg, D. Hopwood, "Recursive Proof Composition without a Trusted Setup," IACR ePrint 2019/1021. Crate used: halo2_proofs 0.3.5.

[13] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger, "Poseidon: A New Hash Function for Zero-Knowledge Proof Systems," USENIX Security 2021. IACR ePrint 2019/458.

[14] zcash/halo2 pull request #964, "Remove unsound SHA-256 Table16 gadget," merged 29 September 2026. https://github.com/zcash/halo2/pull/964

[15] CVE-2026-54496 / GHSA-ww9q-8r59-xv46, soundness flaw in the variable-base scalar multiplication gadget of halo2_gadgets, published 6 July 2026. https://osv.dev/vulnerability/GHSA-ww9q-8r59-xv46

[16] J. Sugiarto, “PCP/0: Proof-Contingent Payment,” specification and reference implementation, 2026. https://github.com/juliansugiarto/pcp (pcp/SPEC.md, pcp-risc0/COSTS.md, pcp-zk/COSTS.md, pcp-zk/review/2026-10-21-failure-report.md)